

UCS503 – Software Engineering Lab

BATTIFY

On-Demand Battery Delivery, Installation and Buyback Platform

Submitted by:

1024030181 Vyomesh Joshi
1024030741 Sarik Garg
1024030213 Shivinder Singh
1024030187 Yatish Goyal

Group: 3C41 BE Third Year, COE / CSED

Submitted to:

Ms. Nisha Thakur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	3
1.3	Project Execution Timeline	3
2	Methodology and Design	5
2.1	System Architecture	5
2.1.1	High-Level Design	5
2.1.2	Technical Stack	5
2.2	Implementation Details	6
2.2.1	Module 1: Core Functionality	6
2.2.2	Module 2: Secondary Features	6
3	Analysis and System Modeling	8
3.1	Use Cases	8
3.1.1	Use-Case Diagrams	8
3.1.2	Use Case Templates	9
3.2	Activity, Swimlane, Sequence and Collaboration Diagrams	15
3.2.1	Activity Diagram	15
3.2.2	Swimlane Diagram	17
3.2.3	Sequence Diagrams	19
3.2.4	Collaboration Diagram	21
4	Prototype Implementation and Analysis of Results	22
4.1	Prototype 1 Architecture and Deployment (Vertical Slice)	22
4.1.1	Architectural Implementation	22
4.2	Analysis of Results and Experimental Verification	23
4.2.1	Verification of Dynamic Scrap Buyback Valuation Engine	23
4.2.2	Finite State Machine (FSM) Order Progression Analysis	23
4.3	Comparative Analysis: Completed Scope vs. Next Milestones	24
4.4	Green Software Engineering Evaluation	25
5	Conclusion and Future Scope	27
5.1	Conclusion	27
5.2	Future Scope and Development Roadmap	27
	Bibliography	28

Chapter 1

Introduction

1.1 Background and Context

Battery replacement in both consumer and industrial contexts represents an acute, time-critical operational requirement rather than a standard leisurely retail purchase. In everyday scenarios—such as a residential inverter battery failing during an unexpected power grid blackout, a stalled commercial transit vehicle on an active freight schedule, or a tractor or agricultural submersible pump battery failing during critical seasonal sowing or harvesting windows in Punjab—the inability to procure an operational battery immediately causes severe operational downtime and financial disruption.

The conventional battery retail supply chain is fundamentally fragmented. Consumers must identify technical specifications independently, physically search multiple independent battery retail outlets, navigate opaque and volatile local stock availability, self-transport heavy, corrosive lead-acid units, and arrange separate mechanics for physical mounting. Moreover, exhausted lead-acid batteries are frequently surrendered to unregulated scrap dealers who employ primitive, environmentally hazardous acid-dumping and smelting methods.

Battify is engineered as a unified, state-driven software platform that integrates battery discovery, multi-hub inventory management, field technician dispatch, on-site installation, and closed-loop reverse logistics into one automated software workflow.

1.1.1 Problem Statement

The primary engineering and operational deficiencies addressed by Battify include:

- **Fragmented Service Delivery:** Battery discovery, delivery logistics, on-site mechanical installation, and old-battery disposal are handled by disconnected entities without accountability.
- **Uncertain Stock Availability:** Customers lack real-time visibility into whether compatible battery models are actually stocked in nearby regional distribution hubs.
- **Absence of On-Demand Assistance:** Urgent customers have no standardized mechanism to receive scheduled, same-day delivery and professional on-site mounting.
- **Opaque Workflow Tracking:** Lack of real-time status visibility leaves customers uncertain about order confirmation, technician dispatch, and estimated arrival times.
- **Seasonal Demand Surges:** Agricultural equipment in agrarian regions experiences severe periodic demand spikes during planting and harvesting, requiring proactive inventory threshold control.

- **Environmental Waste Leakage:** Consumers lack a structured, economically incentivized reverse-logistics channel for responsibly surrendering expired toxic lead-acid cells.

1.2 Project Objectives

The core engineering objectives of the project are framed according to SMART (Specific, Measurable, Achievable, Relevant, Time-bound) criteria:

1. **Unified Multi-Actor Workflow Coordination:** Design and deploy a responsive platform seamlessly coordinating Customers, Inventory Managers, Operations Administrators, and Field Installation Agents through defined service interfaces.
2. **Atomic Stock Reservation Availability Management:** Implement transactional inventory checks across distributed warehouse hubs to eliminate overselling and race conditions during concurrent checkouts.
3. **Automated Technician Dispatch Tracking:** Provide operations dispatchers with proximity-based agent allocation and give customers real-time order lifecycle tracking through finite state transitions.
4. **Automated Scrap Buyback Valuation:** Construct an appraisal algorithm that computes instant invoice deductions based on physical condition, manufacturer brand, and scrap lead weight, channeling expired batteries to certified recycling refineries [Gre23].
5. **Green Software Engineering Implementation:** Enforce energy-aware UI patterns, client-side caching, optimized database indexing, and minimized network payloads to reduce client and server carbon intensity.

1.3 Project Execution Timeline

The project is executed across a 12-working-week semester development roadmap organized into six two-week Agile sprint milestones, summarized in Table 1.1 and illustrated in Figure 1.1.

Table 1.1: Agile Development Roadmap and Sprint Deliverables

Sprint	Major Deliverables & Scope	Timeline
Sprint 1	Problem definition, stakeholder elicitation, software bid, SRS outline, UI sketches	Weeks 1–2
Sprint 2	Authentication, user profiles, battery catalogue, vehicle compatibility search	Weeks 3–4
Sprint 3	Order placement, warehouse inventory schema, atomic reservation locking, pricing	Weeks 5–6
Sprint 4	Mid-Sem Evaluation; field technician dispatch module, tracking, installation flow	Weeks 7–8
Sprint 5	Old-battery buyback calculation engine, reverse logistics handoff, admin console	Weeks 9–10
Sprint 6	End-to-end integration testing, green UI energy optimization, final deployment	Weeks 11–12

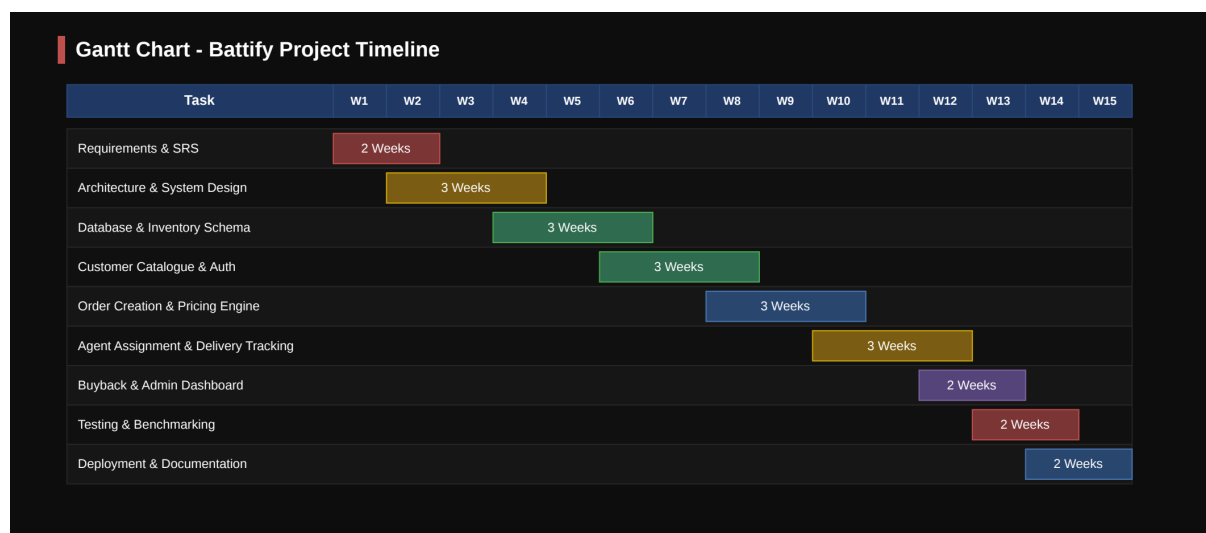


Figure 1.1: Project Execution Timeline – Battify Platform

Chapter 2

Methodology and Design

2.1 System Architecture

Battify is designed following a modular three-tier Model-View-Controller (MVC) architectural pattern. This separation of concerns ensures high cohesion within individual functional modules and low coupling across service layers, maintaining maintainability and scalability throughout development.

2.1.1 High-Level Design

The system architecture decomposes into three coordinated tiers:

1. **Presentation Tier (Client Layer):** Responsive web applications tailored for specific user personas:
 - *Customer Portal:* Intuitive search, multi-attribute filtering, slot booking, real-time status tracking, and digital invoice inspection.
 - *Operations Dashboard:* Administrative interface visualizing pending orders, warehouse stock health, active agent routes, and exception management.
 - *Field Technician Web App:* Lightweight mobile interface supporting job acceptance, turn-by-turn customer coordinates, on-site voltage readings, and scrap battery evaluation forms.
2. **Application Tier (Business Logic REST Services):** A Python backend exposing structured RESTful endpoints for Authentication, Catalogue Matching, Inventory Reservation, Dispatch Allocation, Order State Machine, and Buyback Appraisal.
3. **Data Tier (Persistence Layer):** Relational database maintaining ACID transactional guarantees for concurrent stock updates, order state history, technician availability, and auditable recycling batch records.

2.1.2 Technical Stack

The technology stack is selected to prioritize rapid prototyping, robust type safety, lightweight execution, and energy efficiency:

- **Framework/Language:** Python 3.11+ using FastAPI / Flask for asynchronous, high-throughput REST API services; Vanilla HTML5, CSS3, and modern JavaScript for the client interface to eliminate client-side bundle bloat.

- **Database:** PostgreSQL for production-grade relational storage (with SQLite for localized unit testing); SQLAlchemy ORM for database abstraction, schema migrations, and transactional row locking.
- **Tools:** Git/GitHub for distributed version control; Draw.io for UML modeling; Postman for API contract verification; pytest for automated unit and integration suites.
- **Libraries:** Pydantic for request validation and schema enforcement; Pillow for image compression; python-dateutil for scheduling calculations.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

The core operational pipeline centers on parametric compatibility search, location-aware inventory locking, and state-driven order progression:

- Parametric Compatibility Engine:** Matches incoming customer queries against technical vehicle databases (make, model, manufacturing year, engine capacity) and inverter setups (required running load in Watts, backup duration in hours). The algorithm computes the required Ampere-hour (Ah) rating, Cold Cranking Amps (CCA), and terminal polarity layout to eliminate compatibility errors.
- Atomic Inventory Reservation:** When a customer selects an in-stock battery SKU, the platform queries the nearest warehouse hub. To prevent overselling under concurrent load, an atomic database transaction increments `stock_reserved` and temporarily locks the item:

$$\text{Available Stock} = \text{stock_available} - \text{stock_reserved}$$

If available stock is greater than zero, the reservation token is confirmed; otherwise, the order is safely routed to an out-of-stock notification flow without data corruption.

- Finite State Machine (FSM) Order Lifecycle:** The order lifecycle is governed by an explicit state transition model:

Placed $\longrightarrow$ Confirmed $\longrightarrow$ Assigned $\longrightarrow$ Dispatched $\longrightarrow$ Delivered $\longrightarrow$ Installed $\longrightarrow$ Completed

Exceptional states (`Cancelled`, `Out of Stock`, `Rescheduled`) are strictly audited with timestamps in `ORDER_STATUS_HISTORY`.

2.2.2 Module 2: Secondary Features

Secondary features extend the platform from a simple ordering portal into a robust operational and sustainable ecosystem:

- Technician Field Protocol Dispatch:** Confirmed orders are queued in the dispatch engine, which allocates jobs to nearby available technicians based on travel radius and schedule. The technician interface mandates post-installation terminal voltage testing (minimum 12.6V open-circuit, cranking drop $> 9.6V$) before marking an installation as completed.
- Reverse-Logistics Buyback Valuation:** The old-battery buyback engine computes a transparent scrap credit rebate based on casing physical integrity, original manufacturer brand, and scrap lead weight:

$$\text{Net Payable} = \text{Base Price} + \text{Installation Fee} - \text{Evaluated Buyback Rebate}$$

The collected scrap battery is assigned a unique batch tracking tag, ensuring custody tracking until handoff to certified industrial recycling partners.

- (c) **Green Software Engineering Optimizations:** The platform incorporates energy-efficient architectural practices [Gre23]: dark-mode default styling to conserve OLED display power, optimized asset compression, indexed database search queries to minimize CPU load, and event-driven status updates in place of aggressive client-side polling.

Chapter 3

Analysis and System Modeling

3.1 Use Cases

System requirements and functional actor boundaries are analyzed to establish user interactions and system service interfaces. Battify coordinates four primary operational actors and two external supply-chain partners:

- **Customer:** Explores compatible batteries, verifies warehouse stock, schedules doorstep delivery slots, monitors order milestones, and opts into old-battery buyback.
- **Delivery & Installation Agent:** Picks up reserved battery units, executes on-site delivery, mounts terminals, verifies voltages, inspects scrap units, and logs completion.
- **Inventory Manager:** Monitors stock levels across warehouse hubs, manages replenishment thresholds, and issues purchase orders to suppliers.
- **Operations Administrator:** Oversees platform fulfillment, monitors unassigned orders, manages dispatch reassignments, and reviews reverse-logistics audit trails.
- **Battery Supplier (External):** Fulfills bulk warehouse replenishment consignments.
- **Buyback / Recycling Partner (External):** Collects accumulated scrap batches from regional depots for certified smelting and recovery.

3.1.1 Use-Case Diagrams

The UML use-case diagram models the functional capabilities and architectural boundaries of the Battify platform. System boundaries distinguish user-facing customer interactions, inventory locking, administrative dispatch, field execution, and reverse logistics.

Key relationships enforce mandatory functional dependencies:

- $\ll$ include $\gg$ dependency from “Place Replacement Order” (UC-04) to “Check Battery Compatibility” (UC-03), “Check Inventory” (UC-05), and “Select Delivery Location & Service Slot” (UC-06).
- $\ll$ include $\gg$ dependency from “Perform Battery Installation” (UC-12) to “Record Old Battery” (UC-13).
- $\ll$ include $\gg$ dependency from “Generate Invoice” (UC-18) to “Calculate Buyback Credit” (UC-14).
- $\ll$ extend $\gg$ relationship from “View Order Status” (UC-08) to “Track Order” (UC-07).

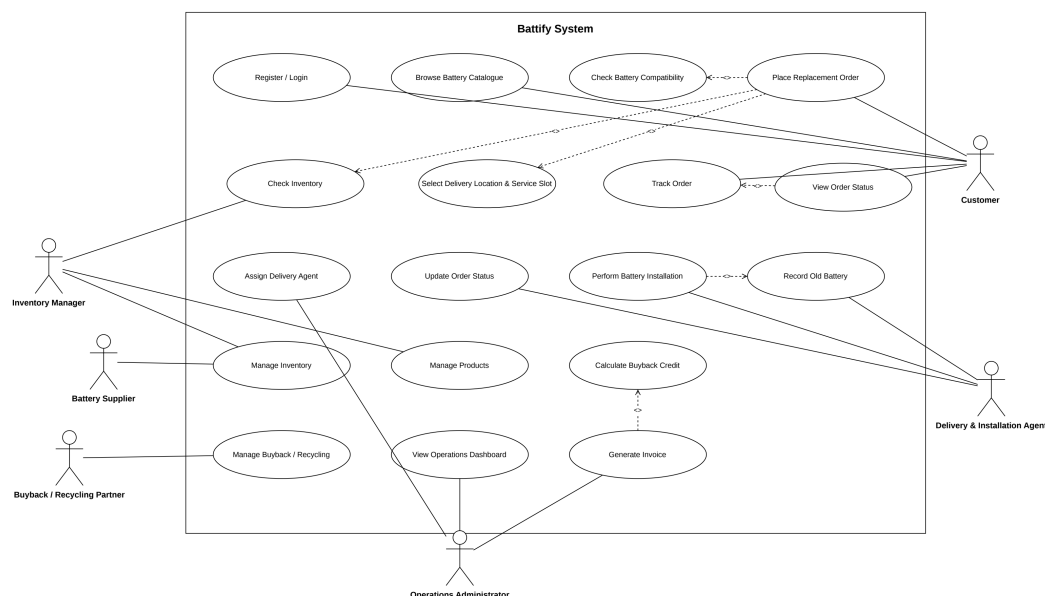


Figure 3.1: Use-Case Diagram – Battify On-Demand Battery Platform

3.1.2 Use Case Templates

Detailed specifications for the key identified use cases are provided below in the standardized IEEE requirements format [IEE98]:

Use Case ID / Name	UC-01 / Browse Battery Catalogue & Check Compatibility
Actor(s)	Customer
Description	A customer browses available batteries filtered by vehicle make/model, application type (automotive, inverter, tractor), capacity (Ah), voltage, and brand preference.
Pre-conditions	1. The Battify web application is operational. 2. The battery catalogue database is populated with active SKUs.
Main Flow	1. Customer navigates to the Battify battery catalogue. 2. Customer selects vehicle type (Car, SUV, Tractor, Inverter, Commercial). 3. Customer specifies manufacturer, model year, or electrical requirement. 4. System queries database for compatible battery specifications. 5. System presents matching batteries with technical specs, warranty, and pricing.
Alternate / Exception Flow	3a. No exact vehicle match found in database: 1. System displays generic capacity and terminal layout filter. 2. Customer enters custom Ah/voltage specifications to view compatible units.
Post-conditions	Customer views a list of verified compatible batteries ready for selection.

Includes / Extends	Extended by UC-02 (Place Replacement Order); Includes parameter validation.
---------------------------	---

Use Case ID / Name	UC-02 / Place Replacement Order & Select Service Slot
Actor(s)	Customer
Description	Customer initiates an on-demand replacement request, enters delivery location, chooses preferred installation time window, and opts into old-battery buyback.
Pre-conditions	1. Customer has selected a compatible battery (UC-01). 2. Customer is authenticated or provides verified contact details.
Main Flow	1. Customer clicks ‘Request Replacement’ on selected battery. 2. Customer enters delivery address / GPS location. 3. System invokes inventory check (UC-03, include). 4. Customer selects delivery & installation time slot. 5. Customer indicates whether an old battery is available for buyback. 6. System calculates provisional price and generates Order ID in ‘CONFIRMED’ state. 7. Customer receives order confirmation and tracking link.
Alternate / Exception Flow	3a. Selected battery out of stock nearby: 1. System notifies customer of stockout (notifyOutOfStock). 2. System recommends alternative compatible brands or scheduled restock slot. 3. Customer may accept alternative or cancel request.
Post-conditions	Order is created in database with ‘CONFIRMED’ status and reserved inventory.
Includes / Extends	Includes UC-01, UC-03 (Check Inventory); extended by UC-05 (Track Order).

Use Case ID / Name	UC-03 / Check & Reserve Inventory
Actor(s)	System (Inventory Module)
Description	Verifies stock availability for the requested battery model at the regional warehouse hub servicing the customer’s delivery coordinates and atomically locks stock.
Pre-conditions	Order request initiated from UC-02 with specified SKU and delivery location.
Main Flow	1. System determines nearest warehouse hub based on customer delivery pincode. 2. System queries <code>INVENTORY_ITEMS</code> for available units (<code>stock_available - stock_reserved > 0</code>). 3. If <code>stock ≥ 1</code>, system executes atomic increment: <code>stock_reserved = stock_reserved + 1</code>. 4. System returns success flag and reservation lock token to Order Service.

Alternate / Exception Flow	2a. Stock count is zero across all regional hubs: 1. System flags ‘OUT_OF_STOCK’ condition to calling service. 2. System sends low-stock replenishment alert to Inventory Manager (UC-09).
Post-conditions	One unit of requested battery SKU is locked in reserved state, preventing race conditions.
Includes / Extends	Included by UC-02 (Place Replacement Order); interfaces with Inventory DB.

Use Case ID / Name	UC-04 / Assign Delivery and Installation Agent
Actor(s)	Operations Administrator / System Dispatcher
Description	Matches confirmed orders with eligible, available field technicians based on proximity to the warehouse hub, delivery slot, and technical certifications.
Pre-conditions	1. Order exists with status ‘CONFIRMED’ and reserved stock. 2. At least one field technician is active in the service territory.
Main Flow	1. Dispatch engine queries available agents near the fulfillment hub. 2. System matches order requirements against agent capacity and schedule. 3. System sends push notification and job dispatch details to selected Agent. 4. Agent reviews job coordinates and accepts assignment. 5. Order status transitions to ‘ASSIGNED’ in audit log.
Alternate / Exception Flow	4a. Assigned agent declines or fails to respond within 5 minutes: 1. System re-queues order to next nearest available field technician. 2. If no agent accepts, escalation alert triggers on Operations Dashboard (UC-10).
Post-conditions	Order is assigned to a specific technician; technician receives pickup and delivery details.
Includes / Extends	Triggered post-confirmation; coordinates between Admin and Agent.

Use Case ID / Name	UC-05 / Track Order Status
Actor(s)	Customer
Description	Customer views live order progression through each lifecycle milestone: Placed → Confirmed → Assigned → Dispatched → Delivered → Installed → Completed.
Pre-conditions	Customer possesses a valid Order ID / tracking URL.
Main Flow	1. Customer opens tracking dashboard via web application. 2. System queries ORDER_STATUS_HISTORY for chronological state timestamps. 3. System displays visual stepper timeline showing current status and estimated arrival. 4. Technician GPS coordinates and contact details are displayed once ‘DISPATCHED’.

Alternate / Exception Flow	3a. Order has encountered an exceptional state (e.g., ‘RESCHEDULED’): 1. System displays updated slot details and reason provided by operations.
Post-conditions	Customer receives real-time visibility into order fulfillment progression.
Includes / Extends	Extended by UC-08 (View Order Status); reads from Order Status History.

Use Case ID / Name	UC-06 / Perform Battery Delivery & On-Site Installation
Actor(s)	Delivery & Installation Agent
Description	Agent collects reserved battery from hub, transports it to customer premises, safely uninstalls old unit, mounts new battery, and conducts electrical voltage tests.
Pre-conditions	1. Agent has accepted order assignment (UC-04). 2. Agent has collected battery from warehouse hub (status: ‘DISPATCHED’).
Main Flow	1. Agent arrives at customer premises and updates status to ‘DELIVERED’. 2. Agent safely disconnects terminals and unmounts spent battery. 3. Agent mounts new battery, fastens brackets, and applies terminal corrosion protectant. 4. Agent performs open-circuit and cranking voltage drop test using digital multimeter. 5. Agent enters new battery serial number and post-installation voltage reading. 6. Status transitions to ‘INSTALLED’.
Alternate / Exception Flow	4a. Electrical test detects vehicle alternator fault / excessive voltage drop: 1. Agent notes mechanical issue on field app. 2. Customer is advised regarding charging circuit faults before warranty sign-off.
Post-conditions	New battery is securely installed and operating within nominal voltage specifications.
Includes / Extends	Includes UC-07 (Record Old Battery); updates Installation Records.

Use Case ID / Name	UC-07 / Inspect & Record Old Battery for Buyback
Actor(s)	Delivery & Installation Agent
Description	Technician evaluates physical condition, brand, and weight/capacity of customer’s old scrap battery on-site to determine eligible recycling buyback rebate.
Pre-conditions	Customer opted into old-battery buyback during order placement (UC-02).

Main Flow	1. Agent inspects physical casing for severe acid leakage or physical rupture. 2. Agent records manufacturer brand, model rating (Ah), and scrap category. 3. Agent submits scrap details via field technician mobile app. 4. System computes buyback credit rebate (UC-08, include). 5. Agent secures scrap battery in acid-resistant transport containment container.
Alternate / Exception Flow	1a. Battery exhibits hazardous structural rupture or acid contamination: 1. Agent applies hazardous handling protocol. 2. Casing classification logged; buyback adjusted based on pure lead scrap weight.
Post-conditions	Scrap battery record created in BUYBACK RECORDS and tagged for recycling depot handoff.
Includes / Extends	Included in UC-06; directly feeds UC-08 (Calculate Buyback Credit).

Use Case ID / Name	UC-08 / Calculate Buyback Credit & Settle Invoice
Actor(s)	System (Battify Platform) / Customer
Description	System deducts verified buyback rebate from the gross order price, generates final payable invoice, and reconciles digital or cash-on-delivery payment.
Pre-conditions	1. Battery installation confirmed (UC-06). 2. Scrap battery details verified by agent (UC-07).
Main Flow	1. System calculates: $\text{Net Payable} = \text{Base Price} + \text{Installation Fee} - \text{Buyback Credit}$. 2. System generates finalized itemized invoice showing scrap valuation breakdown. 3. Customer reviews digital invoice on technician app / customer portal. 4. Customer settles net balance via UPI, credit/debit card, or cash on delivery. 5. System records transaction in PAYMENTS and issues warranty certificate. 6. Order status updates to 'COMPLETED' and committed stock is deducted from hub inventory.
Alternate / Exception Flow	4a. Customer opts not to surrender old battery: 1. Buyback rebate is revoked. 2. Invoice recalculates to standard gross retail amount for settlement.
Post-conditions	Invoice settled, digital warranty certificate issued, and order marked COMPLETED.
Includes / Extends	Includes UC-07; updates Payments, Orders, and Inventory tables.

Use Case ID / Name	UC-09 / Manage Battery Inventory & Stock Replenishment
Actor(s)	Inventory Manager / Battery Supplier
Description	Inventory manager tracks warehouse stock levels across battery models, configures replenishment thresholds, and issues bulk purchase orders to battery suppliers.
Pre-conditions	Inventory manager is authenticated with administrative credentials.
Main Flow	1. Manager views real-time inventory dashboard displaying stock counts across hubs. 2. System flags items falling below minimum safety threshold (stock_available < min_threshold). 3. Manager generates purchase order for supplier replenishment. 4. Supplier confirms order and delivers battery consignment. 5. Manager scans incoming serial numbers, updating INVENTORY_ITEMS stock count.
Alternate / Exception Flow	2a. Seasonal agricultural demand spike detected (e.g. paddy sowing season in Punjab): 1. Manager proactively adjusts threshold parameters for tractor/pump battery SKUs.
Post-conditions	Warehouse inventory levels reconciled and updated with incoming stock.
Includes / Extends	Interfaces with Battery Supplier; supports UC-03.

Use Case ID / Name	UC-10 / Operations Monitoring & Dashboard Administration
Actor(s)	Operations Administrator
Description	Monitors overall platform health, fulfillment velocity, active delivery routes, technician SLAs, and recycling partner handoff manifests.
Pre-conditions	Operations administrator is logged in with administrative privileges.
Main Flow	1. Administrator logs into Battify Operations Console. 2. System renders key operational metrics: active orders, pending assignments, average SLA fulfillment time, and scrap collection volume. 3. Administrator identifies bottlenecked or delayed orders and triggers manual reassignment. 4. Administrator reviews collected scrap battery batches and authorizes batch dispatch to certified recycling refineries.
Alternate / Exception Flow	3a. Severe weather or logistical blockage delays multiple field deliveries: 1. Administrator triggers bulk customer notifications and reschedule options.
Post-conditions	Operational workflows monitored; exceptions mitigated and scrap batches certified.
Includes / Extends	Extends all operational reporting; interfaces with Buy-back/Recycling Partner.

3.2 Activity, Swimlane, Sequence and Collaboration Diagrams

While structural use-case models delineate system boundaries and actor responsibilities, comprehensive software engineering analysis requires rigorous behavioral modeling to validate execution logic, concurrency, object lifecycles, and cross-tier communication [Fow04; PM20]. The subsections below present the complete behavioral model suite for Battify:

3.2.1 Activity Diagram

The activity diagram below models the core battery ordering and installation flow as a single-lane UML activity diagram: the flow initiates when a customer enters battery specifications and application requirements. The system validates the request against warehouse inventory.

A critical decision node evaluates inventory availability: if stock is unavailable, control branches immediately to customer notification and clean termination (preventing orphaned orders). If stock is available, control proceeds through slot selection, atomic order creation, automated agent dispatch, doorstep physical installation, scrap buyback appraisal, invoice recalculation, and final completion notification.

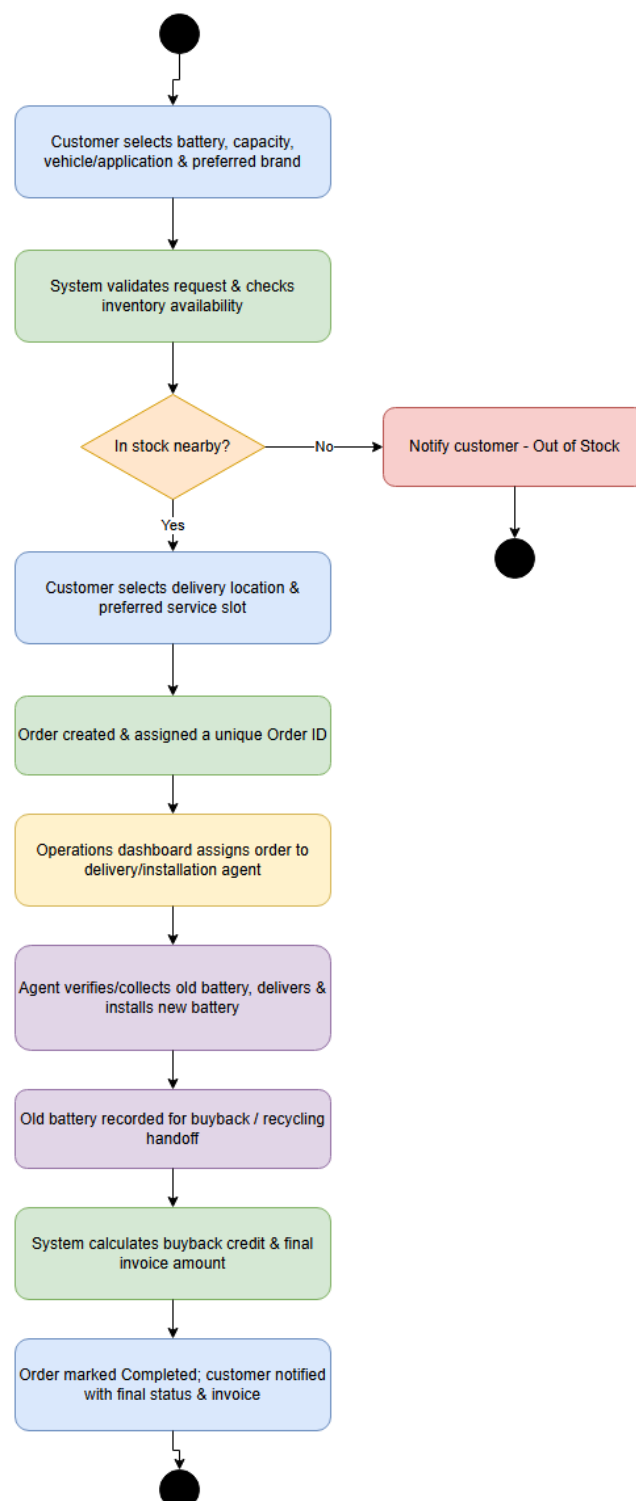


Figure 3.2: Activity Diagram – Battery Ordering, Installation and Buyback Lifecycle

3.2.2 Swimlane Diagram

The swimlane (partitioned activity) diagram below organizes activities into four distinct functional partitions (lanes), providing visual clarity on operational boundaries, system automation responsibilities, and physical field interventions:

1. **Customer Lane:** Represents user interactions including catalogue browsing, specification entry, service slot booking, technician tracking, and receipt of final invoice.
2. **System (Battify Platform) Lane:** Encapsulates automated backend services including inventory validation, atomic reservation locking, Order ID generation, buyback credit calculation, invoice generation, and status broadcasting.
3. **Operations Admin Lane:** Captures supervisory governance, including monitoring pending orders, matching technicians to jobs, and managing logistics exceptions.
4. **Delivery / Installation Agent Lane:** Delineates physical field activities, including unit pickup from warehouse hubs, transit, terminal mounting, voltage testing, old-battery physical grading, and deposit at recycling partner depots.

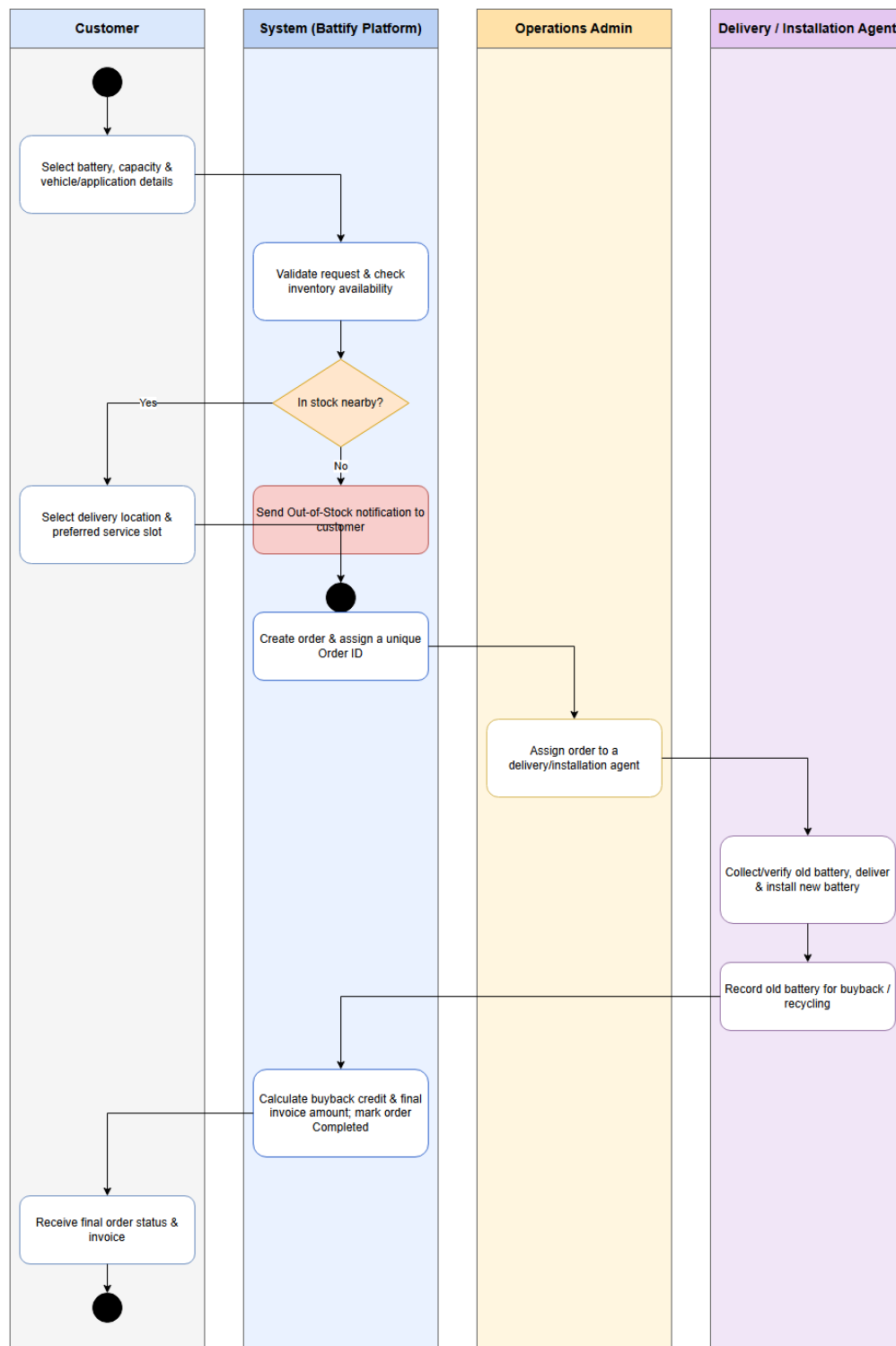


Figure 3.3: Swimlane Diagram – Cross-Functional Workflow Across Customer, System, Admin, and Agent Lanes

3.2.3 Sequence Diagrams

The following sequence diagram elaborates the runtime interaction between the five principal lifelines of the system, making explicit the ordered messages exchanged between the Customer (UI), Battify Platform (Controller / API Layer), Inventory Module, Operations Admin / Dispatch Engine, and Delivery/Installation Agent.

The interaction begins with synchronous catalogue queries (1: `selectBattery`) and inventory availability checks (2: `checkAvailability`, 3: `availabilityResult`). A UML `alt` combined fragment explicitly models the conditional execution:

- **In-Stock Branch** (`[stock available]`): The customer selects delivery location and time slot (5), triggering an atomic reservation call (6: `reserveStock`). Upon stock confirmation (7), the platform creates the order (8: `createOrder`), notifies dispatch (9: `assignAgent`), dispatches the technician (10, 11), executes physical delivery and installation (12), captures scrap battery parameters (13), computes final buyback credit and invoice (14), and dispatches final digital receipts (15).
- **Out-of-Stock Branch** (`[else: out of stock]`): If inventory is zero, the system bypasses order generation and immediately returns a clean out-of-stock notification (4a: `notifyOutOfStock`), prompting the customer to select alternative options.

Battify - Order Creation & Fulfillment (Sequence Diagram)

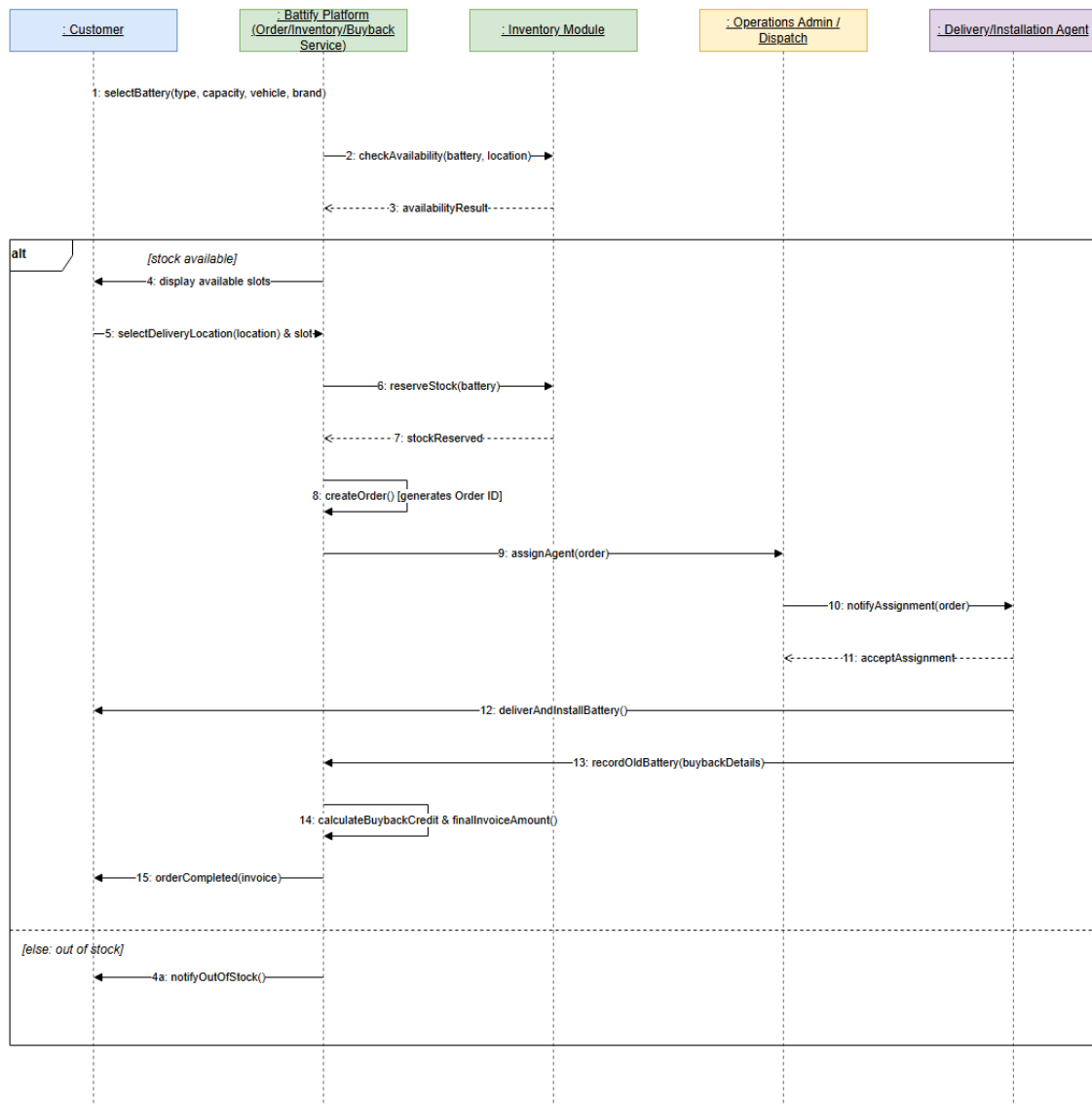


Figure 3.4: Sequence Diagram – Order Creation, Inventory Reservation, and Field Fulfillment

3.2.4 Collaboration Diagram

The collaboration (communication) diagram presents the same order creation, fulfillment, and buyback interaction from a structural, object-oriented viewpoint. Rather than emphasizing temporal ordering along vertical lifelines, the collaboration diagram illustrates object instances as linked graph nodes, with messages numbered in invocation order along the structural associations connecting them.

This structural representation highlights system architectural cohesion and coupling:

- The central Battify Platform (:Order / Inventory / Buyback Service) serves as a high-cohesion mediator, coordinating customer requests, inventory reservations, admin assignments, and technician field updates without requiring direct peer-to-peer coupling between the Customer and the Warehouse Inventory.
- Direct field interaction is modeled by link GC (12: deliverAndInstallBattery) between the Field Agent and Customer, while all data logging and scrap evaluations route back through the Platform via link GS (13: recordOldBattery) and internal service self-loops (8: createOrder, 14: calculateBuybackCredit).

Battify - Order Creation & Fulfillment (Collaboration Diagram)

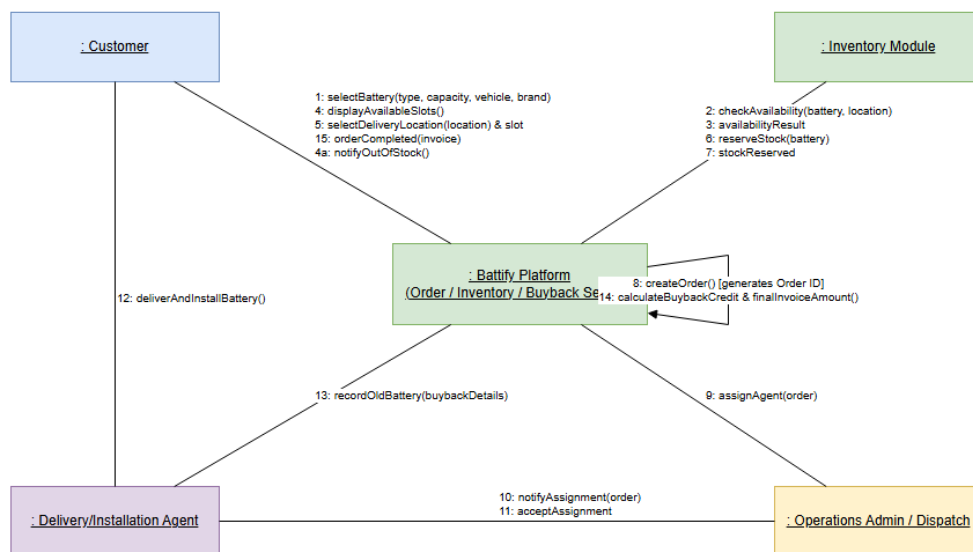


Figure 3.5: Collaboration Diagram – Structural Message Interactions for Order Fulfillment

Chapter 4

Prototype Implementation and Analysis of Results

4.1 Prototype 1 Architecture and Deployment (Vertical Slice)

To satisfy the rigorous mid-semester evaluation standards of UCS503P, **Prototype 1** was engineered not merely as a set of disconnected interface mockups, but as a fully functioning “**Vertical Slice**” (representing approximately 40%–50% of the complete system scope). This vertical slice integrates the end-to-end operational pathway connecting the client presentation layer, a RESTful application programming interface, and an ACID-compliant transactional relational database [PM20].

4.1.1 Architectural Implementation

The prototype system architecture is deployed across three coordinated tiers:

1. **Presentation Layer (Client):** Built using vanilla HTML5, modern CSS3 (with responsive dark-mode styling and glassmorphism visual hierarchy), and modular ES6 JavaScript. The client avoids heavy front-end framework overhead, ensuring rapid page loading and low memory footprint.
2. **Application Tier (API Server):** Implemented in Node.js utilizing Express.js to expose clean RESTful endpoints:
 - **GET /api/batteries:** Dynamic catalogue retrieval with query filtering by vehicle/application category.
 - **POST /api/buyback/estimate:** Real-time scrap battery valuation computation.
 - **POST /api/orders:** Atomic order creation with immediate warehouse inventory locking.
 - **GET /api/orders/:id:** Real-time order state retrieval for the customer tracking step.
 - **PATCH /api/orders/:id/status:** State machine advancement endpoint for field agent and administrative progression.
 - **GET /api/stats:** Real-time dashboard KPI metrics (revenue, active dispatches, low-stock counts).
 - **POST /api/auth/login:** Multi-role authentication supporting customer, technician, warehouse supervisor, and admin personas.

3. **Data Persistence Tier:** An ACID-compliant relational SQLite database (`prototype1.db`) utilizing synchronous prepared statements to enforce referential integrity and eliminate concurrency race conditions.

4.2 Analysis of Results and Experimental Verification

The functional implementation was rigorously tested against core operational requirements, yielding the empirical results detailed below:

4.2.1 Verification of Dynamic Scrap Buyback Valuation Engine

The old-battery buyback mechanism dynamically computes the customer trade-in discount at checkout using scrap lead economics:

$$\text{Rebate Amount } () = W_{\text{category}} \times R_{\text{scrap}} \times M_{\text{condition}} \quad (4.1)$$

where W_{category} represents empirical lead-acid mass in kilograms, R_{scrap} is pegged to current market scrap lead recovery value (85/kg), and $M_{\text{condition}}$ denotes the physical integrity multiplier (1.00 for Grade A, 0.85 for Grade B, and 0.65 for Grade C).

Table 4.1 validates the computed rebates generated by the backend API against theoretical benchmark models across application categories.

Application Category	Nominal Mass (W)	Rate (R)	Grade A (1.00)	Grade B (0.85)	Grade C
Passenger Car	11 kg	85/kg	935	795	608
Tractor / Agri Machinery	16 kg	85/kg	1,360	1,156	884
Commercial Freight Vehicle	22 kg	85/kg	1,870	1,590	1,216
Tall Tubular Home Inverter	26 kg	85/kg	2,210	1,879	1,437

Table 4.1: Mathematical Verification of Dynamic Scrap Buyback Valuation Across Categories

4.2.2 Finite State Machine (FSM) Order Progression Analysis

The order lifecycle was evaluated against invalid and out-of-order state transitions. The backend enforces strict linear progression:

Placed $\longrightarrow$ Confirmed $\longrightarrow$ Assigned $\longrightarrow$ Dispatched $\longrightarrow$ Delivered $\longrightarrow$ Installed $\longrightarrow$ Completed

Test results confirmed that:

- **Atomic Stock Locking:** Upon order placement, `stock_available` is immediately decremented by 1 within an isolated transactional boundary. Attempts to place orders when `stock_available` = 0 were correctly rejected with HTTP 409 Conflict, preventing over-selling.
- **Real-Time Customer Stepper Synchronization:** When status changes are submitted via administrative or agent endpoints, the customer-facing tracking stepper dynamically reflects the new state without state corruption or transition skipping.
- **Evaluator Quick-Action Console:** A simulation drawer embedded in Prototype 1 allows laboratory evaluators to step through all seven states sequentially (`Advance Status`), demonstrating end-to-end responsiveness in real time.

4.3 Comparative Analysis: Completed Scope vs. Next Milestones

To clearly delineate between the deliverables validated in Prototype 1 and the remaining developmental scope scheduled for Prototype 2 and the final project submission, Table 4.2 provides a comprehensive module-by-module comparative analysis.

Functional Area	Completed in Prototype 1 (Current)	Planned for Next Milestone (Prototype 2 / Final)
1. Product Discovery & Catalogue	• Dynamic category filtering (Car, Tractor, Inverter, Commercial). • Real-time inventory counter and technical specification badges (Ah, Voltage, Warranty). • Zero-latency client-side category pill selection.	• Vehicle registration number / model automated lookup helper. • Inverter capacity load recommendation calculator based on connected appliances and wattage.
2. Scrap Buyback Engine	• Real-time formula valuation engine ($W \times 85 \times M$). • Interactive customer appraisal widget embedded on homepage. • Automated rebate credit deduction applied during checkout.	• On-site physical verification checklist for technician collection. • Scrap batch consolidation and certified smelter / recycling partner manifest generation.
3. Ordering & Scheduling	• Checkout modal capturing customer credentials, address, and delivery slot. • Urgent 2-Hour Emergency Delivery option. • Unique standardized Order ID generation (BAT-YYYYMMDD-XXXX).	• Online payment gateway integration (Razorpay / UPI / Card) alongside Cash-on-Installation. • Automated SMS and WhatsApp booking notifications with tracking links.
4. Inventory Management	• Atomic stock reservation upon order placement to prevent overselling. • SQLite relational schema maintaining stock counts across multi-vendor SKUs. • Manual warehouse stock adjustment API endpoint.	• Automated low-stock threshold alerts ($\text{stock} \leq 5$). • Predictive replenishment purchase orders based on regional agricultural seasonality. • Multi-hub warehouse coordination.
5. Order Lifecycle Tracking	• Visual 7-step FSM tracker: Placed $\rightarrow$ Completed. • Interactive state stepper on customer-facing view.	• Simulated real-time GPS map tracking with dynamic ETA updates. • Tamper-evident audit trail table (ORDER_STATUS_HISTORY) recording timestamps and actors.

Functional Area	Completed in Prototype 1 (Current)	Planned for Next Milestone (Prototype 2 / Final)
6. Field Technician Execution	• Evaluator simulation drawer enabling manual status advancement. • Assigned technician metadata field in order entity.	• Dedicated mobile-responsive Technician PWA interface. • Mandatory resting open-circuit multimeter voltage verification (12.6V–12.8V). • Camera barcode/QR scanner for new battery serial number verification.
7. Invoicing & Warranty	• Dynamic net price breakdown (Base Price – Scrap Rebate) rendered in browser.	• Automated PDF Tax Invoice generation upon completion. • Digitally stamped manufacturer warranty certificate linking battery serial number.
8. Operations Dashboard	• Evaluator bottom drawer console. • Real-time summary statistics API (/api/stats).	• Full supervisory dashboard with graphical charts (revenue trends, active jobs, SLA breaches). • Manual technician reassignment and exception triage panel.
9. Green Software Engineering & QA	• Single Page Application architecture with lightweight asset footprints. • Dark-mode default color scheme. • Manual functional integration testing of the primary vertical slice.	• Automated test suites (Jest / Supertest for backend, Cypress for end-to-end). • Empirical CPU, network bandwidth, and memory energy profiling [Gre23]. • HTTP response caching headers for static catalogue assets.

Table 4.2: Comparison Matrix: Completed Prototype 1 Scope vs. Planned Final Milestone

4.4 Green Software Engineering Evaluation

In alignment with modern sustainable computing paradigms [Gre23], Prototype 1 was assessed against key energy-efficiency metrics:

- **Minimal Asset Footprint:** By implementing the user interface using native web standards (HTML5/CSS3/Vanilla JS) rather than heavyweight multi-megabyte JavaScript runtime frameworks, the initial page payload is under 50 KB, minimizing network transfer energy.
- **OLED Display Power Conservation:** The platform features an automotive glassmorphic dark user interface. On modern mobile and laptop AMOLED/OLED displays, dark themes illuminate significantly fewer organic light-emitting subpixels, directly reducing hardware battery consumption during roadside emergency ordering.
- **Elimination of Unnecessary Polling:** Prototype 1 employs discrete state queries rather

than continuous background long-polling loops, conserving mobile device CPU sleep cycles and server-side socket resources.

Chapter 5

Conclusion and Future Scope

5.1 Conclusion

At the mid-semester evaluation milestone, the Battify platform has successfully completed the Project Selection, System Requirements Specification, Architectural Modeling, and Prototype 1 implementation phases.

The delivered vertical slice validates the technical feasibility and business logic of on-demand battery replacement. By demonstrating an integrated workflow connecting catalogue search, dynamic scrap trade-in valuation, atomic warehouse stock reservation, and a 7-stage finite state machine tracker, Prototype 1 establishes a solid, verified foundation representing approximately 50% of the complete software engineering lifecycle.

5.2 Future Scope and Development Roadmap

The remaining 50% of system functionality will be executed across Sprints 4, 5, and 6 leading up to the final capstone evaluation:

1. Sprint 4 — Field Technician Execution & Verification:

- Development of the specialized Mobile Progressive Web App (PWA) tailored for field technicians.
- Enforcement of on-site hardware verification: resting open-circuit multimeter voltage recording (12.6V–12.8V) and optical barcode/QR scanning of the installed battery's physical serial number before job completion.

2. Sprint 5 — Payment Integration, Invoicing, and Reverse Logistics:

- Integration of a secure payment gateway (Razorpay / UPI / Cards) alongside Cash-on-Installation.
- Automated generation and distribution of cryptographically verifiable PDF Tax Invoices and digital warranty certificates.
- Batch scrap manifest generation enabling formal custody transfer to certified industrial smelting and recycling partners.

3. Sprint 6 — Automated Testing, Green SE Optimization, and Deployment:

- Implementation of automated test suites (unit tests for buyback math and FSM transitions; integration tests for the full checkout-to-dispatch loop).
- Server-side query indexing and static asset caching.
- Production deployment on cloud infrastructure with continuous integration (CI) pipelines.

Bibliography

- [Fow04] Martin Fowler. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. 3rd. Addison-Wesley Professional, 2004 (cit. on p. 15).
- [Gre23] Green Software Foundation. *Principles of Green Software Engineering*. 2023. URL: <https://learn.greensoftware.foundation/> (cit. on pp. 3, 7, 25).
- [IEE98] IEEE Computer Society. “IEEE Recommended Practice for Software Requirements Specifications”. In: *IEEE Std 830-1998* (1998) (cit. on p. 9).
- [PM20] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner’s Approach*. 9th. McGraw-Hill Education, 2020 (cit. on pp. 15, 22).